Payment Gateway Service

Single Design Document

Cloud-Native High Availability Architecture

Document ID:	SDD-PGW-2026-002
Version:	1.8
Date:	April 2, 2026
Classification:	Internal - Confidential
Owner:	Cloud Engineering Team
Status:	Approved

Version History

Version	Date	Author	Changes
1.8	04/02/2026	M. Chen	Updated scaling limits for Q2 2026
1.7	02/15/2026	M. Chen	Added X-Ray tracing configuration
1.6	12/01/2025	S. Patel	DR region warm standby setup
1.5	09/20/2025	M. Chen	Migration from EKS to ECS Fargate
1.0	03/01/2025	S. Patel	Initial document creation

1 Purpose of this Document

The purpose of this document is to provide a comprehensive reference architecture for the Payment Gateway Service (PGW), Example Organization's unified payment processing platform. The PGW handles all digital payment transactions including card-present, card-not-present, peer-to-peer transfers, real-time payments (RTP), and wire transfers across all banking channels.

This document captures the cloud-native architecture deployed on Amazon Web Services, the high availability design leveraging multi-AZ deployment, the auto-scaling strategy validated for 12,000 transactions per second, and the disaster recovery approach using Aurora Global Database. It serves as the authoritative reference during architecture reviews, compliance assessments, and operational planning.

The intended audience includes architecture review boards, cloud engineering teams, security assessors (particularly PCI-DSS QSAs), disaster recovery planners, and operations personnel responsible for maintaining the Payment Gateway in production. The document assumes familiarity with AWS services, container orchestration concepts, and payment processing terminology.

1.1 Business Context

The Payment Gateway Service processes approximately 22 million transactions daily with a combined value exceeding $4.8 billion. It serves as the single entry point for all payment channels and routes transactions to the appropriate processing backend based on payment type, card network, and regulatory requirements. Peak capacity was validated at 12,000 transactions per second during Black Friday 2025 with zero degradation in latency or error rates.

The service supports seven distinct payment types, each with unique processing requirements, network routing, regulatory obligations, and latency expectations. Card-present transactions (POS) require sub-2-second end-to-end latency to avoid customer-facing timeouts. Real-time payments (RTP via The Clearing House) must complete within 3 seconds per network rules. ACH and wire transfers operate on batch windows defined by the Federal Reserve schedule.

The migration from a legacy on-premises deployment to AWS was completed in September 2025, driven by three business imperatives: elasticity (the ability to scale 5x for shopping events without pre-provisioning hardware), operational efficiency (eliminating server patching, capacity management, and hardware lifecycle), and speed-to-market (reducing the deployment cycle from weekly to multiple times per day).

1.2 Design Philosophy

The Payment Gateway architecture is built on cloud-native principles that differ fundamentally from the on-premises deployment it replaced. Every design decision prioritizes operational simplicity, automatic recovery, and observable behavior over raw performance optimization or cost minimization. The philosophy can be summarized in four principles:

Managed Services First: Use AWS managed services (ECS Fargate, Aurora, SQS, ElastiCache) rather than self-managed equivalents whenever the managed service meets functional requirements. This trades some configurability for dramatically reduced operational burden and built-in HA.

Stateless Compute: Application containers are completely stateless and disposable. All state lives in Aurora (durable), Redis (ephemeral/cacheable), or SQS (transient/queued). Any container can be terminated at any time without data loss or processing interruption.

Fail Fast, Recover Automatically: Circuit breakers prevent cascading failures by failing fast when downstream services are unhealthy. ECS automatically replaces failed tasks. Aurora automatically promotes readers. Every failure mode has an automatic recovery path.

Observe Everything: Every request is traced (X-Ray), every metric is captured (CloudWatch custom metrics at 1-second resolution), every log is structured and correlated. The system is designed to be debuggable in production without SSH access to containers.

1.3 Payment Types Supported

Payment Type	Daily Volume	Avg Amount	Latency SLA	Network
Card Present (POS)	8.2M	47 USD	< 2 seconds	Visa/MC/Amex
Card Not Present (e-comm)	5.1M	83 USD	< 3 seconds	Visa/MC/Amex
P2P Transfer (Zelle)	2.3M	215 USD	< 5 seconds	Zelle Network
Real-Time Payment (RTP)	1.8M	1,240 USD	< 3 seconds	TCH RTP
ACH (Same-day)	3.5M	2,100 USD	< 30 minutes	Nacha
Wire Transfer	450K	28,000 USD	< 30 minutes	FedWire/SWIFT
Bill Payment	1.2M	340 USD	< 5 seconds	Various billers

2 High Level Architecture

2.1 Architecture Overview

The Payment Gateway is deployed as a set of containerized Python microservices running on ECS Fargate across three Availability Zones in us-east-1. The architecture follows a synchronous request-response pattern for payment authorization (where latency is critical) and an asynchronous event-driven pattern for downstream processing (where reliability and ordering are more important than speed).

Traffic enters through CloudFront (edge caching for static content and DDoS protection via AWS Shield Advanced) and AWS WAF (OWASP Top 10 rules, custom rate limiting, and geographic blocking for OFAC-sanctioned countries). API Gateway provides request validation, API key management, and throttling. The Application Load Balancer distributes requests across ECS tasks in all three AZs using health-check-based routing.

The data layer consists of Aurora PostgreSQL (transactional data store with Multi-AZ deployment and Global Database for cross-region DR), ElastiCache Redis (idempotency keys, session cache, circuit breaker state), and SQS/EventBridge (asynchronous event processing for notifications, reconciliation, and analytics). All data stores are encrypted at rest with KMS Customer Master Keys and in transit with TLS 1.2 minimum.

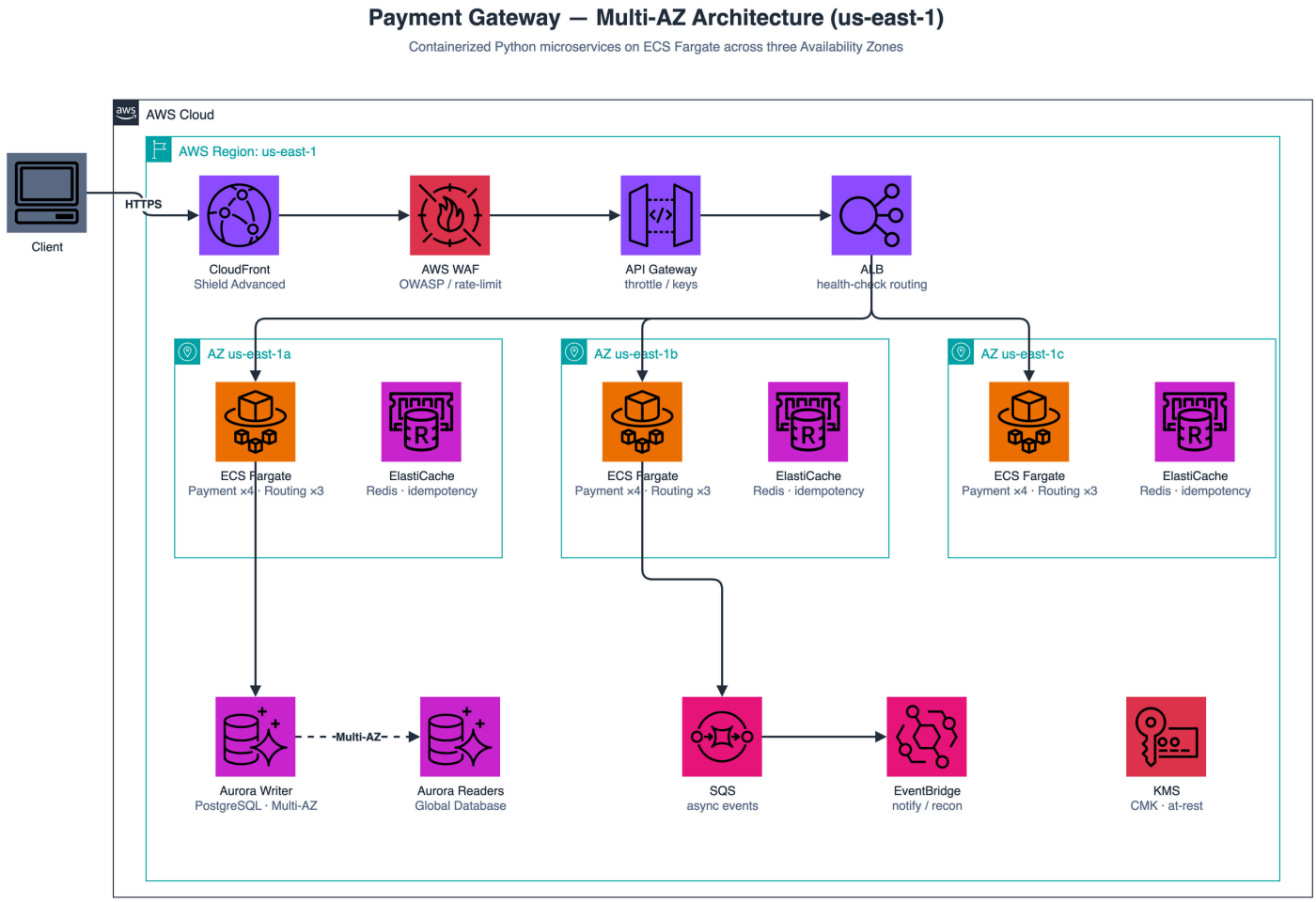


Figure 1: Payment Gateway - Multi-AZ Architecture in us-east-1

2.2 Service Architecture

The Payment Gateway consists of six microservices, each deployed as an independent ECS service with dedicated auto-scaling policies, IAM roles (least privilege), health check endpoints, and circuit breaker configurations. Services communicate synchronously via HTTP (within the authorization path) and asynchronously via SQS/EventBridge (for downstream processing). Each service owns its scaling behavior independently.

Service	Language	Tasks	Responsibility
payment-service	Python 3.11 / FastAPI	12-60	Payment orchestration, validation, idempotency
routing-service	Python 3.11 / FastAPI	8-40	Network routing, scheme detection, fallback logic
tokenization-service	Python 3.11 / FastAPI	6-24	PAN tokenization, PCI scope isolation
settlement-service	Python 3.11 / FastAPI	4-16	EOD settlement batch, reconciliation matching
notification-worker	Python 3.11 / SQS	4-20	Payment confirmations (SMS, push, email)
fraud-connector	Python 3.11 / FastAPI	6-24	Real-time fraud scoring API integration

2.3 Technology Selection Rationale

Each major technology choice was evaluated against alternatives with documented decision records. ECS Fargate was selected over EKS to minimize operational complexity -- the team did not need Kubernetes-level flexibility (no service mesh, no custom scheduling, no sidecar patterns) and valued Fargate serverless container model that eliminates all node management, patching, and capacity planning at the compute layer.

Aurora PostgreSQL was chosen over DynamoDB for its SQL flexibility, multi-table ACID transactions, and the Aurora Global Database capability for cross-region DR with sub-second replication lag. DynamoDB was considered for its infinite scalability but rejected because payment processing requires complex queries (transaction history, reconciliation matching) and multi-item transactions that DynamoDB handles poorly.

SQS and EventBridge were selected over self-managed Kafka (MSK) for downstream event processing. For the Payment Gateway use cases (notifications, reconciliation, analytics feed), SQS provides sufficient guarantees (at-least-once delivery, 14-day retention, dead-letter queues) with zero operational overhead. The decision explicitly traded Kafka replay capability for operational simplicity, with the rationale that EventBridge event replay covers the most common replay scenarios.

3 High Availability Design

3.1 Multi-AZ Architecture

All components of the Payment Gateway are distributed across three Availability Zones (us-east-1a, 1b, 1c) with no zone affinity or preference. The architecture is designed to sustain the complete loss of any single AZ without any degradation in service quality, capacity, or latency. This is achieved by provisioning sufficient capacity in the remaining two AZs to handle the full production load -- a design choice that trades cost efficiency for resilience.

ECS Fargate tasks are distributed evenly across AZs through the ECS spread placement strategy. The ALB uses cross-zone load balancing to ensure even distribution regardless of the number of healthy targets in each AZ. Aurora PostgreSQL maintains reader instances in different AZs from the writer, with automatic failover promoting a reader in a healthy AZ if the writer AZ fails. ElastiCache Redis cluster mode distributes shards across all three AZs with replica promotion on node failure.

The design explicitly avoids any AZ-specific configuration or state that would make one AZ special. There are no primary or preferred AZs -- the system operates identically regardless of which AZ is lost. This symmetry simplifies operations and eliminates a class of failure modes related to asymmetric configuration.

3.2 Auto-Scaling Strategy

Each ECS service is configured with target-tracking auto-scaling policies that respond to both CPU utilization and request count metrics. The dual-metric approach prevents situations where a single metric might not capture the true load. For example, an I/O-bound workload (waiting for card network responses) may not show high CPU but still needs more instances to handle concurrent requests without exhausting connection pools.

Scale-out is aggressive (60-second cooldown) to respond quickly to traffic spikes, while scale-in is conservative (300-second cooldown) to avoid oscillation during variable load patterns. Scheduled scaling pre-warms the system for predictable traffic patterns: weekday business hours (increase minimum to 18 tasks at 8AM ET, reduce at 8PM ET), month-end processing (24 task minimum for last 2 business days), and planned shopping events (36 task minimum, manually activated 24 hours before).

Service	Metric	Target	Scale-Out	Scale-In
payment-service	CPU Utilization	60%	60s cooldown	300s cooldown
payment-service	ALB Request Count/Target	1500	60s cooldown	300s cooldown
routing-service	CPU Utilization	60%	60s cooldown	300s cooldown
tokenization-svc	ALB Request Count/Target	1000	60s cooldown	300s cooldown
settlement-svc	SQS ApproximateMessages	100	60s cooldown	300s cooldown
notification-worker	SQS ApproximateMessages	500	60s cooldown	300s cooldown

3.3 Failure Scenarios & Validated Recovery

The architecture has been validated against a comprehensive set of failure scenarios through both automated chaos engineering experiments (weekly GameDay exercises) and manual disaster recovery tests (quarterly). Each scenario

documents the expected detection time, automated response, validated recovery time, and any data loss:

Scenario	Detection	Recovery Action	RTO	RPO
Single ECS task crash	ALB health check (10s)	ECS replaces task automatically	30 seconds	0
Multiple tasks in 1 AZ	ALB + CW composite alarm	ECS spreads to remaining AZs	30 seconds	0
Complete AZ failure	ALB cross-zone routing	2 AZs absorb full load	60 seconds	0
Aurora writer failure	Aurora health detection	Auto-promote reader to writer	30 seconds	0
Redis primary failure	Cluster mode detection	Replica automatic promotion	15 seconds	0
Full region failure	Route53 health check (90s)	Failover to us-west-2	5 minutes	< 1 min
Card network timeout	Circuit breaker (5 failures)	STIP local processing	Immediate	0
DNS resolution failure	Synthetic monitoring	CloudFront cached + fallback	60 seconds	0

4 Network Architecture

4.1 VPC Design

The Payment Gateway VPC (10.100.0.0/16) is designed with strict network segmentation following defense-in-depth principles. Four subnet tiers isolate components by function and access requirements, with security groups enforcing least-privilege communication between tiers. No component in the data tier is directly accessible from any public network; all access flows through the ALB in the public subnets and is subject to WAF inspection.

The subnet architecture follows AWS best practices for high availability: each tier has one subnet per AZ (3 subnets per tier), with route tables configured to direct internet-bound traffic through per-AZ NAT Gateways (eliminating cross-AZ NAT traffic and providing AZ-level isolation for egress). The /24 subnet size provides 251 usable IP addresses per subnet, sufficient for the maximum ECS task count with significant headroom.

Tier	Subnets	CIDR Range	Components	Internet Access
Public	3 (one per AZ)	10.100.1-3.0/24	ALB nodes, NAT Gateways	Direct (IGW)
Private - App	3 (one per AZ)	10.100.11-13.0/24	ECS Fargate tasks	Outbound only (NAT)
Private - Data	3 (one per AZ)	10.100.21-23.0/24	Aurora, ElastiCache	None
Private - Mgmt	2	10.100.31-32.0/24	VPC Endpoints	None

4.2 Security Groups

Security groups implement a whitelist-only model where each component can only communicate with its direct dependencies on specific ports. The security group chain creates a strict layered architecture: Internet -> ALB (443) -> ECS tasks (8080) -> Aurora (5432) / Redis (6379). No lateral movement is possible between unrelated components, and no component can initiate outbound connections except to its declared dependencies and VPC endpoints.

Security group rules reference other security groups by ID rather than IP ranges wherever possible. This ensures that rules remain correct even as IP addresses change (e.g., when ECS tasks are replaced). The only CIDR-based rules are the ALB inbound rule (0.0.0.0/0 on 443 for public access) and the VPC endpoint prefix list references.

4.3 VPC Endpoints (PrivateLink)

All AWS service access is routed through Interface VPC Endpoints, ensuring that no application traffic traverses the public internet. This is a PCI-DSS Requirement 1 control for systems handling cardholder data -- all communication with AWS services stays within the AWS network backbone without internet exposure. The following endpoints are provisioned:

- com.amazonaws.us-east-1.sqs -- SQS message operations

- cor

5 Data Flow & Event Architecture

5.1 Payment Authorization Flow

The payment authorization flow is the latency-critical synchronous path that processes a customer payment request from initiation to approval or decline. The total latency budget is 2000ms at P99, though typical transactions complete in 180-350ms depending on the card network. Each step has independent timeout and circuit breaker configurations to prevent cascading failures from a single slow dependency.

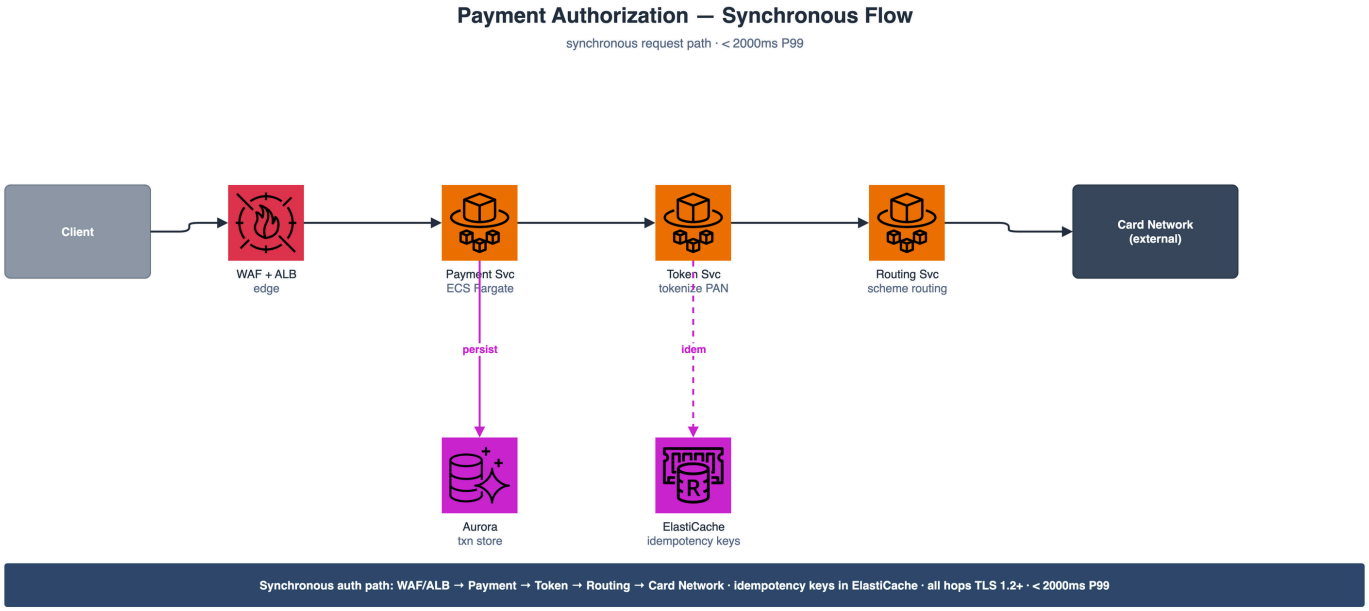


Figure 2: Payment Authorization Synchronous Flow

The flow begins when a client application sends an HTTPS POST request containing payment details (amount, currency, card token or raw PAN, merchant information, and channel identifier). CloudFront passes the request to API Gateway which validates the request schema against an OpenAPI specification, checks the API key, and applies per-client throttling. The request then reaches the ALB which selects a healthy payment-service instance based on the least-outstanding-requests algorithm.

The payment-service orchestrates the authorization by calling dependent services in an optimized order: first, it checks Redis for an idempotency key (using the client-provided Idempotency-Key header) to detect and deduplicate retry attempts. If no duplicate is found, it calls the tokenization-service in parallel with the fraud-connector to minimize sequential latency. Once both respond, it calls the routing-service which determines the appropriate card network and sends the authorization request.

The routing-service maintains per-network configuration including endpoint URLs, timeout values (Visa: 5s, Mastercard: 5s, RTP: 3s), retry policies (at most 1 retry for network timeouts, no retry for explicit declines), and circuit breaker thresholds (5 failures in 10 seconds opens the circuit). When a circuit is open, the routing-service immediately returns a soft decline rather than waiting for inevitable timeouts, protecting system resources and providing fast feedback to the customer.

5.2 Asynchronous Event Processing

After a payment is completed, the payment-service emits a PaymentCompleted event to Amazon EventBridge using the PutEvents API. EventBridge evaluates the event against configured rules and routes it to multiple SQS queues simultaneously, enabling independent downstream processing without coupling consumers to the payment-service or to each other. This fan-out pattern is critical for extensibility -- new consumers can be added by creating a new EventBridge rule and SQS queue without modifying any existing code.

Current event consumers include: the notification-worker (sends payment confirmations via SMS, push notification, or email within 30 seconds of payment completion), the reconciliation service (matches authorizations with settlements for end-of-day balancing), the fraud analytics pipeline (feeds transaction data to the ML model training system), and a Kinesis Data Firehose stream that delivers events to the S3 data lake in near-real-time for long-term analytics.

Each consumer operates independently with its own error handling, retry logic, and DLQ. If one consumer fails (e.g., the notification service is down), it does not impact other consumers or the primary payment processing flow. Messages accumulate in the failed consumer SQS queue and are processed when the consumer recovers, providing natural backpressure without data loss.

5.3 Dead Letter Queue Strategy

Every SQS queue has a corresponding Dead Letter Queue (DLQ) configured with a maxReceiveCount of 3 (notification queues) or 5 (reconciliation queues). Messages that fail processing after the maximum receive count are moved to the DLQ for manual investigation. All DLQs have CloudWatch alarms that fire on the first message arrival -- a non-empty DLQ always indicates a problem requiring investigation, even a single message.

DLQ messages are retained for 14 days, providing sufficient time for investigation and reprocessing. A Lambda function runs daily to generate a DLQ health report summarizing message counts, age distribution, and common error patterns. Reprocessing is performed through a dedicated CLI tool that moves messages back to the source queue with optional filtering and transformation.

6 Database & Caching

6.1 Aurora PostgreSQL Configuration

The primary data store is an Aurora PostgreSQL 15.4 cluster configured with a writer instance (db.r6g.2xlarge: 8 vCPUs, 64GB RAM) in AZ-1a and two reader instances (db.r6g.xlarge: 4 vCPUs, 32GB RAM) in AZ-1b and AZ-1c respectively. Aurora was selected for its combination of PostgreSQL wire-protocol compatibility (enabling standard PostgreSQL drivers and ORM tooling), automatic storage scaling (no pre-provisioning required), and the Global Database feature for cross-region DR.

Aurora storage provides 6-way replication across 3 AZs with a 4/6 write quorum and 3/6 read quorum. This storage architecture means the database can sustain the loss of an entire AZ plus one additional storage segment while continuing to serve both reads and writes without any operator intervention. The storage layer self-heals by reconstructing lost copies from remaining healthy segments, returning to full redundancy within minutes.

Key Aurora configuration parameters have been tuned for the payment processing workload: shared_buffers is set to 75% of instance RAM (48GB on the writer), effective_cache_size reflects the full memory (64GB), work_mem is increased to 64MB to support complex reconciliation queries without spilling to disk, and idle_in_transaction_session_timeout is set to 30 seconds to prevent connection pool exhaustion from orphaned transactions.

6.2 ElastiCache Redis

ElastiCache Redis 7.0 serves three distinct functions in the Payment Gateway architecture, each with different consistency and durability requirements:

Idempotency Keys: Each payment request carries a client-generated Idempotency-Key header. The payment-service stores this key in Redis with a 24-hour TTL before processing. If the key already exists, the stored result is returned immediately without re-processing. This prevents duplicate charges during client retries or network issues. Loss of Redis data is safe -- it means some duplicate detection is temporarily unavailable, not that duplicates will be created (the database provides final deduplication).

Circuit Breaker State: Circuit breaker states (open/closed/half-open) for each card network are stored in Redis as shared state across all payment-service instances. This ensures that all instances see the same circuit state -- if one instance detects Visa timeouts and opens the circuit, all other instances immediately stop routing to Visa. Without shared state, each instance would need to independently detect the failure, potentially sending hundreds of additional failing requests.

Configuration Cache: Merchant routing configurations, fee schedules, and network-specific parameters are cached in Redis with a 5-minute TTL, reducing database load for these frequently-accessed but rarely-changing data elements. Cache invalidation is triggered by configuration updates through the admin API which publishes an invalidation message to a Redis pub/sub channel.

6.3 Connection Management

Database connections use IAM Database Authentication exclusively, eliminating long-lived password credentials. Each ECS task assumes its service-specific IAM role and generates a short-lived authentication token (15-minute lifetime) for

database connections. This approach eliminates credential rotation as an operational concern, integrates with CloudTrail for connection audit logging, and reduces the blast radius of a compromised container (the token cannot be used from a different IP or role).

7 Disaster Recovery

7.1 Cross-Region Strategy

The disaster recovery strategy uses a warm standby approach in us-west-2, maintaining 30% of production ECS capacity in a ready state with auto-scaling configured to reach 100% within 3 minutes of failover activation. The warm standby represents a deliberate cost/recovery tradeoff: maintaining 30% capacity costs approximately $12K/month but enables a 5-minute RTO, compared to a cold standby that would cost near-zero but require 30+ minutes to scale from nothing.

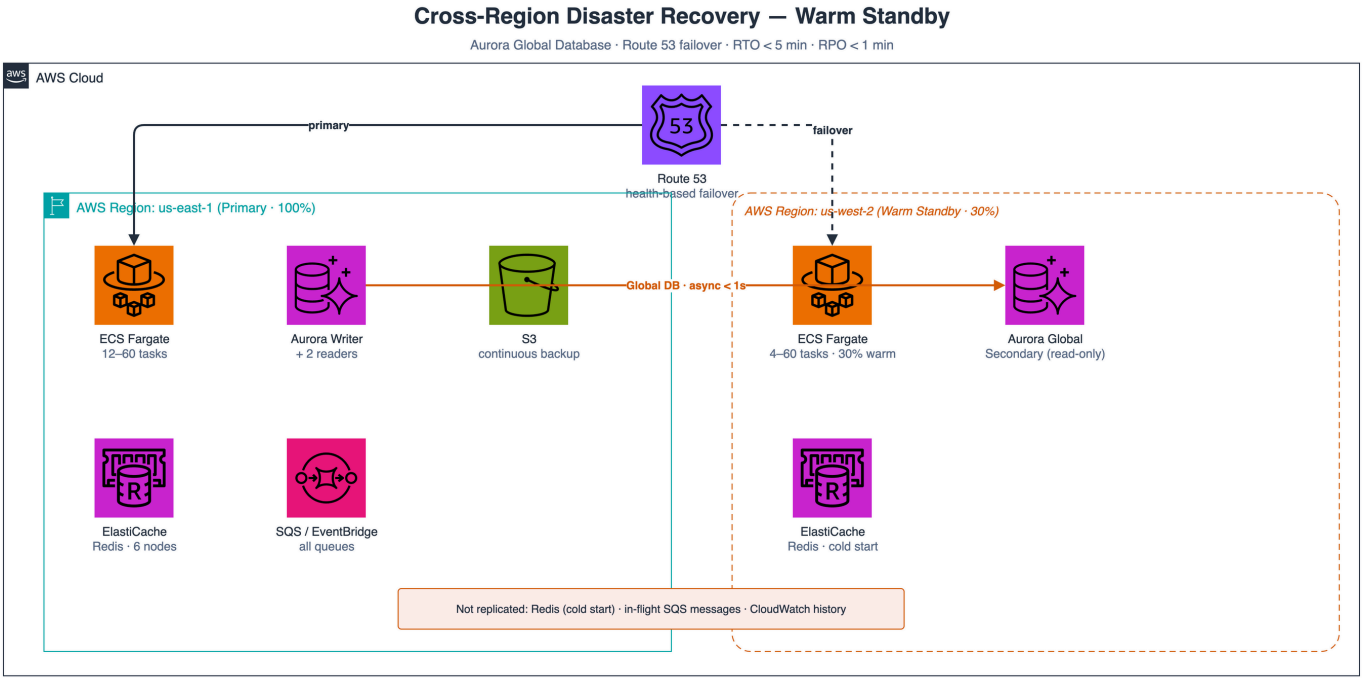


Figure 3: Cross-Region Disaster Recovery Architecture

Aurora Global Database provides the data replication foundation, continuously shipping transaction log records from the primary region to the secondary with measured lag of typically 400-800ms (P99: under 1 second). During failover, the secondary cluster is promoted to a standalone regional cluster capable of accepting writes. This promotion is managed entirely by Aurora and completes in under 1 minute.

Components NOT replicated cross-region include: Redis (cold start from Aurora on failover), SQS queues (messages in-flight at failover time may be lost; idempotent consumers handle replay safely), and CloudWatch metric history (monitoring continuity is maintained through a multi-region Grafana deployment). These gaps are accepted because their impact during failover is limited to slightly elevated latency (Redis warming) or potential message reprocessing (SQS), neither of which affects payment correctness.

7.2 Failover Procedure

Regional failover is semi-automated, requiring human approval at a single gate before execution proceeds automatically. The approval exists because cross-region failover carries non-zero consequences: up to 1 second of committed transactions may need reconciliation (RPO), the failback procedure requires several hours of

re-synchronization, and the DR region has not been validated at sustained production load for more than the quarterly test duration.

The procedure follows these steps: (1) Route 53 health checks detect us-east-1 unavailability -- 3 consecutive failures over 90 seconds trigger a PagerDuty P1 alert. (2) On-call engineer validates the failure is regional, not endpoint-specific. (3) Engineer approves failover in the runbook automation tool, acknowledging the RPO. (4) Aurora Global Database promotes the us-west-2 secondary (< 1 minute). (5) Route 53 weighted record is updated to 100% us-west-2. (6) ECS auto-scaling provisions full capacity (~ 3 minutes). (7) Synthetic transaction validation confirms the DR region is serving correctly.

7.3 DR Testing Results

Full regional failover is tested quarterly. The most recent test (January 15, 2026) achieved the following results: Aurora promotion completed in 48 seconds, ECS reached full capacity in 3 minutes 12 seconds, first successful synthetic transaction at 4 minutes 5 seconds (within 5-minute RTO target), P99 latency stabilized at 380ms (vs. normal 350ms) within 7 minutes as Redis warmed. No payment failures were observed during the test.

8 Security & Compliance

8.1 PCI-DSS Level 1 Controls

As a system processing cardholder data, the Payment Gateway maintains PCI-DSS Level 1 compliance validated through annual assessment by a Qualified Security Assessor (QSA). The architecture implements all 12 PCI-DSS requirement categories with particular emphasis on network segmentation (Req 1), data protection through tokenization (Req 3), encryption in transit (Req 4), and comprehensive logging (Req 10).

The tokenization service is the key architectural control for PCI scope reduction. PANs received from payment requests are immediately tokenized using a format-preserving encryption scheme within an isolated ECS service with its own dedicated KMS key. The original PAN is only re-materialized within the routing-service at the moment of card network transmission, in an ephemeral memory buffer that is zeroed after use. No PAN is ever written to disk, logged, or stored in the database.

8.2 Encryption Architecture

All data is encrypted both in transit and at rest using AWS KMS Customer Master Keys with annual automatic rotation. The encryption architecture uses separate CMKs per service (payment, routing, tokenization) and per data type (database, cache, queue, logs) to support independent access policies and minimize the blast radius of a key compromise.

Resource	Method	Key	Rotation
Aurora PostgreSQL	AES-256 via KMS	CMK: pgw-aurora-key	Annual (auto)
ElastiCache Redis	AES-256 via KMS	CMK: pgw-redis-key	Annual (auto)
SQS Messages	AES-256 via KMS	CMK: pgw-sqs-key	Annual (auto)
S3 Objects	SSE-KMS (AES-256)	CMK: pgw-s3-key	Annual (auto)
CloudWatch Logs	AES-256 via KMS	CMK: pgw-logs-key	Annual (auto)
Secrets Manager	AES-256 via KMS	AWS managed key	Automatic

8.3 WAF & DDoS Protection

AWS WAF v2 is attached to the Application Load Balancer with a layered rule set providing defense against application-layer attacks. Rules are evaluated in priority order: AWS Managed Rules (Common Rule Set, Known Bad Inputs, SQLi Rule Set) block recognized attack patterns, followed by custom rate limiting rules (global: 10,000 requests per 5 minutes per IP; per-endpoint: 1,000 per minute per IP), geographic blocking for OFAC-sanctioned countries, an IP reputation blocklist fed from threat intelligence, and AWS Bot Control for automated traffic management.

DDoS protection is provided by AWS Shield Advanced, which includes automatic detection and mitigation of volumetric attacks at the network layer (L3/L4), cost protection for scaling events triggered by DDoS, and access to the AWS Shield Response Team (SRT) for manual intervention during sophisticated application-layer attacks.

9 Monitoring & Observability

9.1 Three Pillars of Observability

The observability architecture provides comprehensive visibility through three complementary signals: metrics (what is happening at aggregate level), logs (what happened for a specific event), and traces (how a single request flowed through the system). Together, these enable both real-time alerting on anomalies and rapid root-cause investigation during incidents.

Metrics: CloudWatch custom metrics with 1-second resolution for business-critical measurements (payment latency, error rate, TPS). Standard metrics (CPU, memory, network) at 60-second resolution. Custom EMF (Embedded Metric Format) logs enable dimension-rich metrics without separate PutMetricData calls.

Logs: Structured JSON logging with correlation IDs (trace-id, transaction-id, customer-id) enables cross-service log correlation. Logs flow to CloudWatch Logs with 90-day retention, then archive to S3 for long-term (1-year) compliance retention. CloudWatch Logs Insights provides interactive query capability for operational investigation.

Traces: AWS X-Ray distributed tracing samples 5% of normal requests and 100% of error responses. Traces propagate through W3C TraceContext headers across all service boundaries. Custom annotations (transaction_id, payment_type, card_network) enable targeted trace retrieval during incident investigation.

9.2 Critical Alarms

Alarm	Threshold	Evaluation	Action
ALB 5xx Error Rate	> 1% of requests	1 min x 3 datapoints	PagerDuty P1 + auto bridge
P99 Latency Breach	> 2000ms	1 min x 5 datapoints	PagerDuty P1
ECS High CPU (service	> 80%	3 min x 3 datapoints	Auto-scale + PagerDuty P2
Aurora CPU	> 90%	5 min x 3 datapoints	PagerDuty P2
Aurora Replication Lag	> 1000ms	1 min x 5 datapoints	PagerDuty P1
DLQ Non-Empty (any)	> 0 messages	1 min x 1 datapoint	PagerDuty P2
Failed Transactions/min	> 200	1 min x 3 datapoints	PagerDuty P1
Circuit Breaker Open (any)	State = OPEN	Immediate	PagerDuty P2 + Slack

10 Deployment Strategy

10.1 CI/CD Pipeline

The deployment pipeline uses AWS CodePipeline with CodeBuild for a fully automated path from code commit to production deployment. The pipeline enforces quality gates at each stage: unit tests must pass with 90%+ coverage, integration tests against LocalStack must pass, security scans must find zero critical/high findings, and the staging deployment must pass automated smoke tests before production deployment is offered for approval.

The production deployment gate is the only manual step -- a designated approver reviews the change summary, deployment plan, and test results before authorizing the release. This approval can be provided via the CodePipeline console or a Slack integration that presents the deployment summary with approve/reject buttons. Outside of business hours, the approval requirement is relaxed for changes that only modify non-critical paths (e.g., notification templates).

10.2 Rolling Update with Canary

Production deployments use ECS rolling updates with a canary validation phase. The deployment proceeds in three stages: first, new task definitions are registered and 10% of traffic is routed to new containers (canary phase, 5 minutes). During the canary phase, CloudWatch monitors error rate and latency specifically for the new containers via deployment-tagged metrics. If metrics remain healthy, the rolling update continues replacing remaining containers in batches of 25%. If canary metrics breach thresholds, automatic rollback is triggered.

The ECS deployment circuit breaker provides an additional safety net: if 3 consecutive task launches fail health checks, the entire deployment is automatically rolled back to the previous task definition version. This catches issues that only manifest under real traffic (e.g., database schema mismatches, missing secrets) that passed in staging.

10.3 Database Schema Changes

Database schema changes follow an expand-contract pattern to ensure zero-downtime deployments and instant rollback capability. Additive changes (new columns, new tables, new indexes) are deployed in the expand phase before the application code that uses them. Destructive changes (column removal, type changes) are deployed in the contract phase only after confirming that no running application version references the old schema. This guarantees that rollback of application code never requires a corresponding database rollback.

11 Capacity Planning & Performance

11.1 Current Performance

Operation	P50	P95	P99	Target	Status
Card authorization (Visa)	180ms	350ms	800ms	< 2000ms	PASS
Card authorization (MC)	195ms	380ms	850ms	< 2000ms	PASS
P2P transfer (Zelle)	220ms	450ms	950ms	< 2000ms	PASS
Real-Time Payment (RTP)	150ms	280ms	600ms	< 1000ms	PASS
Tokenization (encrypt)	12ms	25ms	45ms	< 100ms	PASS
Balance check (Redis hit)	3ms	8ms	15ms	< 50ms	PASS
Balance check (DB fallback)	8ms	20ms	45ms	< 100ms	PASS

11.2 Load Testing Validation

Quarterly load testing validates that the system maintains acceptable performance under 3x peak load (18,000 TPS). The most recent test (February 2026) confirmed: auto-scaling reached maximum capacity (60 payment-service tasks) within 4 minutes of load onset; P99 latency remained below 450ms at 18,000 TPS; Aurora writer CPU peaked at 78% (below the 85% critical threshold); zero errors were observed through the entire 30-minute sustained peak period.

Beyond 60 tasks, the Aurora writer becomes the bottleneck at approximately 22,000 write TPS. The growth projection (15% year-over-year volume increase) indicates this ceiling will not be reached within 24 months. When approaching this limit, the mitigation path is Aurora instance class upgrade (db.r6g.4xlarge: 16 vCPU, doubling write capacity) followed by write-sharding across multiple Aurora clusters if volume exceeds single-instance capability.

12 Known Risks & Deviations

R-001: DR region at 30% steady-state capacity: The warm standby in us-west-2 runs at 30% of production capacity to optimize cost ($12K/month vs $40K/month for full capacity). Auto-scaling to 100% takes approximately 3 minutes during which early requests may experience slightly elevated latency as the system scales. Quarterly DR tests validate this behavior consistently. Status: Accepted (ARB-2025-167).

R-002: No multi-region active-active database: Aurora Global Database operates in active-passive mode (writes only to primary region). True active-active would eliminate the cross-region failover step but introduces write conflict resolution complexity incompatible with financial transaction semantics. Evaluated CockroachDB and Spanner; rejected due to operational complexity and write-path latency overhead. Status: Accepted (ARB-2025-089).

R-003: Redis not replicated cross-region: ElastiCache Redis exists only in us-east-1. During failover, Redis starts cold and warms from Aurora over 2-5 minutes. Impact: elevated database load and slightly higher latency during cache warming. Idempotency detection is temporarily reduced. Status: Accepted (ARB-2025-112).

R-004: Card network dependency (external): Card network response times are entirely outside the organization control. A persistently slow network (e.g., Visa responding in 4.9s) could consume thread pool resources. Mitigation: per-network circuit breakers with 5-second timeout, independent thread pools per network, and STIP (Stand-In Processing) capability for offline authorization. Status: Implemented.

R-005: SQS regional scope limitation: SQS queues are regional and not replicated cross-region. Messages in-flight at the moment of regional failover may be lost. Mitigation: all consumers are idempotent (safe reprocessing), and EventBridge event replay can regenerate events from source if needed. No payment authorization is affected (synchronous path). Status: Accepted (ARB-2025-134).

13 References

- AWS Well-Architected Framework: Reliability Pillar (2025 edition)

- Am